

# Login System

## Comparison Report

Task 3 — Express.js, MongoDB, Sessions & Class

---

**Student Name:**

Moiz Ansari

---

**Student ID:**

23i-0523

---

**Date:**

6<sup>th</sup> March, 2026

---

# 1. Introduction

---

This report compares two implementations of a Login System built using Express.js, MongoDB (Mongoose), express-session, and JavaScript Classes. The first implementation is written by the student, and the second is generated using an AI tool, that is Gemini in this case.

The purpose of this comparison is to analyze differences in code structure, readability, security practices, and session handling between human-written and AI-generated code.

## 2. Project Overview

---

The login system implements the following features:

- User Registration — saves new users to MongoDB
- User Login — verifies credentials and creates a session
- Protected Dashboard — accessible only to logged-in users
- Logout — destroys the session

### Technologies used:

- Node.js + Express.js — server and routing
- MongoDB + Mongoose — database and schema
- express-session — session management
- JavaScript Class — User class with register() and login() methods

## 3. Student Code

---

### 3.1 Description

I kept basic folder structure like kept the router and all imports in server.js, user class in one file, separate database file and one UserModel file which handles all the mongoose.

I handled all specific edge cases and the conditions according to which the error message will be shown which can be seen in the code below.

I also used environment variables to keep my Mongo Atlas password safe.

### 3.2 File Structure

```
login-system/
├── server.js      ← routes + session + middleware
├── User.js        ← User class (register + login)
├── UserModel.js   ← Mongoose schema + model
├── db.js          ← MongoDB connection
├── index.html     ← HTML Front-end
├── style.css      ← CSS style file
└── package.json
```

### 3.3 Key Code Snippets

#### User Class — register() method:

```
async login() {
  try {
    const exists = await UserModel.findOne({username: this.username});
    if (!exists) {
      return {success: false, message: `Username \"${this.username}\" doesn't
exist.`};
    }
    const user = await UserModel.findOne({username: this.username, password:
this.password});
    if (!user) {
      return {success: false, message: 'Incorrect password'};
    }
    return {success: true, message: "Login successful", username:
this.username};
  }
  catch (error) {
    return {success: false, message: 'Login failed'};
  }
}

async register() {
  if (!this.username || !this.password)
    return { success: false, message: 'Missing fields' };

  const existing = await UserModel.findOne({ username: this.username });
  if (existing)
    return { success: false, message: 'Username already exists' };

  const newUser = new UserModel({ username: this.username, password:
this.password });
  await newUser.save();
  return { success: true, message: 'User registered successfully' };
}
```

## 4. AI Generated Code

---

### 4.1 AI Tool Used

Gemini Pro

### 4.2 Prompt Used

```
[
give me complete code for this assignment
you can have multiple files too just give codes of them separately (IF MORE THAN
ONE FILE)
this will be ur folder structure:
|node_modules
|.env
|.gitignore
|package-lock.json
```

```
package.json
├── GEMINI Version
│   ├── (you files)
│   └──
```

your files will be located inside a folder named "GEMINI Version" placed inside this folder

inside .env file will be two things

MONGO\_URI=(mongo atlas id)

SESSION\_SECRET=(my secret id)

(Login\_System\_Class\_Task.pdf) attached

### 4.3 Description

Gemini structured the code well. It setup the files in separate respective folders which will enable scalability for extending this to large scale project.

It also handled error messages but not as precise as I did. It just shows general error message on missing inputs, etc.

### 4.4 Key Code Snippets

**Paste the AI-generated User class or key parts of server.js here:**

```
async register() {
  try {
    // Check if user already exists
    const existingUser = await UserModel.findOne({ username: this.username });
    if (existingUser) {
      return { success: false, message: 'Username already taken' };
    }

    // Save user to MongoDB
    const newUser = new UserModel({
      username: this.username,
      password: this.password
    });
    await newUser.save();

    return { success: true, message: 'User registered successfully' };
  } catch (error) {
    return { success: false, message: error.message };
  }
}

async login() {
  try {
    // Check user from MongoDB
    const user = await UserModel.findOne({ username: this.username, password:
this.password });
    if (user) {
      return { success: true, message: 'Login successful' };
    } else {
      return { success: false, message: 'Invalid username or password' };
    }
  } catch (error) {
    return { success: false, message: error.message };
  }
}
```

```
}
```

## 5. Comparison Table

Fill in the table below based on your analysis of both implementations:

| Feature           | Student Code                 | AI Code  |
|-------------------|------------------------------|----------|
| Structure         | Basic                        | Advanced |
| Readability       | Easy                         | Easy     |
| Security          | Basic (Didn't hash password) | Basic    |
| Session Handling  | Yes                          | Yes      |
| Error Handling    | Good                         | Good     |
| Code Organization | Advanced                     | Advanced |
| Input Validation  | Yes (Partial)                | No       |

## 6. Detailed Analysis

### 6.1 Structure

The structure of student's code is also neat and in separate files, but Gemini's separate folder structure provides more scalability.

The student's code also has front-end interface to test out the implementation easily.

### 6.2 Readability

The readability of both codes is good. In student's code can be found some neat header comments to easily find out sections of code.

### 6.3 Security

Neither implementation used bcrypt for password hashing. Errors are handles well in both codes. But I used .env file for storing my MongoDB Atlas password string to keep it safe.

### 6.4 Session Handling

Both implementations handle sessions. But Gemini's code does not set sessions expiry date, and student's code does set it to one hour.

The student's code automatically sets information related to logged in session on the front-end.

## 7. Conclusion

---

Both approaches were almost the same. The folder and file structure of Gemini was better. But the error handling was more precise in student's code. Also front-end was added in student's code as an extra feature and easy testing.

### Key takeaways:

- Folder structure should be done by AI
- Basic code from AI, then add specific, precise detailing in the code
- Added extra features in that then

## 8. GitHub Repository

---

### Repository Link:

<https://github.com/muhammad-moiz-ansari/Web-Programming-Tasks>

### Commit History Summary:

```
git commit -m 'Added GEMINI Version'
git commit -m 'Added automatic session loading'
git commit -m 'Added Frontend (HTML + CSS)'
git commit -m 'Added Cookie Sessions'
git commit -m 'Created User.js & setup it in server.js'
git commit -m 'Added Login + Signup Backend Functionality in server.js'
git commit -m 'Added UserModel.js'
git commit -m 'Setup Secure Online MongoDB Atlas Connection'
git commit -m 'Added Local Connection Settings'
git commit -m 'Added db.js'
git commit -m 'Fixed error in routes arguments'
git commit -m 'Added Gitignore File'
git commit -m 'Added Middleware (Protected GET) + Parameter ID'
git commit -m 'Tested Express Session with Modularizarion'
git commit -m 'Setup Basic Express Server'
git commit -m 'Installed Express + Mongoose + express-session'
git commit -m 'TASK 3: Document'
```